IoT-Based Environmental Air Quality Monitoring — Literature Review & Project Scoping

Analysis of 4 IEEE Papers (2023–2025) for IoT Networks, Architectures and Applications Course Project

> **Note on method:** Every factual claim below is drawn strictly from the four uploaded papers. Where the papers are silent, ambiguous, or internally inconsistent, this is flagged explicitly (e.g., "Not specified in the paper" or "Paper inconsistency"). Sections that go beyond direct paper statements are labeled **[Inference]**.

Papers analyzed:

1. **P1 – Vayu Mitra**: "IoT-Enabled Low-Cost Air Quality Monitoring System Using MQ and PM Sensors" (2025 IEEE ICCCMLA)
2. **P2 – Hybrid**: "Integrated IoT-Based Air Quality Monitoring and Prediction System: A Hybrid Approach" (2023 IEEE iSES)
3. **P3 – ACHTECO**: "Air Quality Checking and Tracking for Environmental Compliance: An IoT-based Air Quality Monitoring System" (2025 IEEE ICT&KE)
4. **P4 – Advanced AQMS**: "Advanced Air Quality Monitoring System using IoT and Sensor Technology" (2025 IEEE IDCIoT)

---

1. Executive Summary

All four papers implement the same basic pattern: **low-cost gas/PM sensors → a Wi-Fi microcontroller (ESP32/ESP8266) → a cloud dashboard (ThingSpeak/Blynk/custom web app) → threshold-based alerting**. None of the four papers implement **real-time, on-device ML integrated with live IoT streaming data**. Only P2 uses machine learning at all, and even there it is trained and evaluated **offline on exported CSV data**, not embedded in the live pipeline. None use LoRa, MQTT, ZigBee, or edge inferencing; none report a scalable multi-node/multi-gateway deployment; none use a public benchmark air-quality dataset; and only P3 performs rigorous, quantified system validation (accuracy %, paired t-test, GPS-error testing, stress testing).

The clearest, best-evidenced research gap across all four papers is: **lack of integration between real-time IoT sensor streaming and on-device/edge intelligent prediction or anomaly detection**, compounded by **small-scale, short-duration, single/few-node deployments** and **absence of any public or multi-site dataset**. This is a strong, literature-supported gap that is realistically implementable by a 5-person student team in a semester using Wokwi/Proteus simulation plus a lightweight ML model (e.g., regression or a simple LSTM) trained on a public dataset (e.g., UCI Air Quality, or a Kaggle AQI dataset) and fused with simulated live sensor streams — addressing "monitoring vs. prediction" and "cloud vs. edge" gaps simultaneously.

---

2. Individual Paper Analysis

P1 — Vayu Mitra (2025, ICCCMLA)

Authors: D. Vijaya Saradhi, Pinapati Samyuktha, Shaik Juveriya Khatoon, Shaik Nishath Sameer, Minnekanti Sitha, Shaik Naseema (Malineni Lakshmaiah Women's Engineering College, AP, India)

Field	Detail
Research problem	Conventional AQ stations are accurate but expensive, stationary, and spatially limited, especially in semi-urban/rural areas; need a low-cost, portable, multi-parameter alternative.
Application/domain	Community-level environmental air quality monitoring
IoT architecture	Sensors → Microcontroller (data acquisition + basic filtering + threshold check) → Cloud (ThingSpeak) + Mobile app (Blynk) → SMS/push notification → User
Sensors/hardware	MQ-135 (NH_3, NOx, benzene, CO_2, smoke), MQ-131 (O_3), MQ-7 (CO), PMS7003 (PM1, PM2.5, PM10), DHT11 (temperature/humidity)
Microcontroller/edge	**Inconsistent in the paper**: abstract and methodology state NodeMCU **ESP8266**; the "Proposed Solution" architecture section and Fig. 1 state **ESP32**. The paper does not resolve this contradiction.
Communication protocol	Wi-Fi (to ThingSpeak/Blynk); the architecture text also mentions "Wi-Fi/Bluetooth module" without specifying Bluetooth's role
Data collection method	Continuous polling of all sensors, threshold check on-device, upload to cloud
Dataset	Self-collected, real-time only; no dataset name, no public dataset, no stated collection duration or sample count
Cloud/fog/edge	Cloud only: ThingSpeak (storage/analytics) + Blynk (mobile dashboard/alerts)
AI/ML	**None.** Explicitly listed as a limitation of the (existing) system: "no predictive analysis or AI-based data processing"
What it monitors	CO, CO_2, NH_3, NOx, O_3, benzene, smoke, PM1/PM2.5/PM10, temperature, humidity; threshold-based hazard classification
Evaluation metrics	Not specified — no accuracy, precision, or error metric reported in the provided text
Main results	Qualitative only: system reportedly monitors pollutants "with reliable accuracy" and issues timely alerts; no quantitative results table given
Main contribution	Integration of 3 MQ gas sensors + PM sensor + dual cloud/app dashboards + SMS/push alerting in one low-cost platform
Limitations (explicit)	MQ sensor cross-sensitivity and drift; PM sensing absent in the "existing" (baseline) version they extend; no predictive/AI analysis; alert-only control (no actuation); small-scale operation; Wi-Fi dependency
Future work (explicit)	NDIR-based CO_2 sensors, optical-grade PM sensors, edge computing to reduce cloud dependency, predictive analytics, GIS-based air-quality mapping for smart cities

P2 — Integrated IoT-Based AQ Monitoring and Prediction: Hybrid Approach (2023, iSES)

Authors: Aryan Samal, Lopamudra Samal, Ayas Kanta Swain, KamalaKanta Mahapatra (NIT Rourkela, India)

Field	Detail
Research problem	Air pollution causes ~7 million deaths/year (WHO, as cited); need combined monitoring **and** prediction using both stationary and mobile IoT sensors, which the authors state is "a less-discussed topic"
Application/domain	Smart cities and businesses
IoT architecture	Sensors → Raspberry Pi 3B+ (ADC via MCP3008, filtering, feature extraction) → local DB or cloud (ThingSpeak) → offline ML training (Random Forest Regression) on a client-side PC → trained model deployed back into "IoT device's software system" → mobile app (MIT App Inventor)
Sensors/hardware	DHT11 (temp/humidity; DHT22 also named once, inconsistently), MQ-6 (CO, LPG, methane, alcohol), Honeywell HPMA115S0 (PM2.5, PM10); MCP3008 used as ADC since Raspberry Pi lacks analog pins
Microcontroller/edge	Raspberry Pi 3B+ (quad-core Cortex-A53, 1.4 GHz) — a single-board computer, not a low-power MCU
Communication protocol	Wi-Fi (Raspberry Pi to internet/cloud)
Data collection method	Continuous sampling at defined intervals from both stationary and mobile sensor nodes; data filtered/aggregated before storage
Dataset	Self-collected, real-world, exported from ThingSpeak; **not a public/named dataset**; split 80/20 train/test for RFR
Cloud/fog/edge	ThingSpeak used as the database; processing distributed between Raspberry Pi and cloud "depending on the complexity of the algorithms and available computational resources"
AI/ML	**Random Forest Regression (RFR)**, trained offline on a client-side PC using the exported ThingSpeak dataset
What it predicts	Temperature, humidity, PM2.5, PM10, CO, LPG, methane, alcohol (each modeled as dependent variable in turn, others as features)
Evaluation metrics	RMSE (Root Mean Square Error)
Main results	RMSE for LPG, methane, CO, and alcohol predictions was "considerably big," ranging from 16.535 (lowest) to 29.507 (highest); authors attribute this to "huge abnormality" in the data causing decline in system accuracy
Main contribution	Combining stationary + mobile sensor nodes for improved spatial coverage/accuracy; applying ML prediction to IoT-collected AQ data; cloud + mobile app integration
Limitations (explicit)	High/unstable RMSE for several gas predictions; authors note this indicates data abnormality reduces accuracy — no root-cause fix given
Future work (explicit)	**Not specified in the paper —** no distinct future-work subsection or itemized future directions are given in the provided text; the conclusion restates contributions rather than proposing next steps

P3 — ACHTECO (2025, ICT&KE)

Authors: Leonnel dela Paz de Mesa, Jason Rey Buce Miran, Ivan Kenneth Arevalo Alvarez, Reiner Gabrielle Requina Valdez, Brian Xavier del Rio Hipolito (Mapúa Malayan Colleges Laguna, Cabuyao City, Philippines)

Field	Detail
Research problem	Cabuyao City (Laguna, Philippines) has no official Air Quality Monitoring Station despite Philippine Clean Air Act (RA 8749) mandates; province of Laguna has only 2 AQMS for multiple cities. Need a low-cost, localized, compliance-oriented monitoring system.
Application/domain	Environmental regulatory compliance / local government (CENRO) decision support
IoT architecture	Sensors (MQ2, MQ7, MQ135, DHT22) + NEO-6MV2 GPS → ESP32 → centralized web-hosted database → web application (map, history, information, about, admin pages) → public + government (CENRO) users
Sensors/hardware	MQ2 (SO_2), MQ7 (CO), MQ135 (NO_2, CO_2), DHT22 (temperature/humidity), NEO-6MV2 GPS module
Microcontroller/edge	Primarily stated as **ESP32**; however the Conclusion states data was transmitted "from the ESP8266 microcontroller" — an internal inconsistency similar to P1's.
Communication protocol	Wi-Fi (device connected to Wi-Fi network during 48-hour stress test); no LPWAN/cellular protocol used
Data collection method	Automated readings every 10 minutes over a 24-hour cycle (144 readings/session); GPS-verified location per reading
Dataset	Self-collected, real-world, from 2 physical sensor units deployed at 2 locations in Cabuyao City; not public, no named dataset
Cloud/fog/edge	Centralized, web-hosted database (no named commercial IoT cloud platform like ThingSpeak/Blynk — custom backend)
AI/ML	**None.** AQI is computed using the deterministic US EPA linear-interpolation formula (Eq. 10), not a learned model
What it monitors/detects	CO, CO_2, SO_2, NO_2, temperature, humidity, geolocation; computed Air Quality Index (AQI)
Evaluation metrics	GPS distance error (Haversine formula) vs. Google Earth; accuracy % vs. third-party industrial sensing device; paired t-test (t-statistic) vs. third-party readings; 48-hour continuous stress test; User Acceptance Testing (UAT) rating out of 4
Main results	99.39%–99.97% sensor accuracy vs. professional-grade device; GPS error 8.7–15.5 m (within 20 m threshold, "Acceptable"); paired t-test showed **no significant difference** vs. third-party readings for CO, CO_2, SO_2, NO_2; system stable through 48-hr stress test; UAT scores: citizens 3.43/4, CENRO officials 3.37/4; field SO_2 readings were 0.001–0.002 ppm above the accepted ambient guideline at both test locations
Main contribution	Rigorous, quantitatively validated (accuracy, t-test, GPS-error, stress-test) low-cost AQMS explicitly tied to a real regulatory/legal framework (Clean Air Act), with a full public + admin web platform and Agile Scrum development process
Limitations (explicit)	No PM sensor despite PM being named as a key pollutant of concern in their own literature review; CENRO feedback flagged tooltip clarity and data-summary presentation as needing improvement; SO_2 readings occasionally exceeded ambient guideline at both sites (system correctly detected this, but it indicates real air-quality concern the system doesn't act on beyond reporting)
Future work (explicit)	Fixing bugs affecting data accuracy/UI/responsiveness; UX/navigation enhancements; customized AQI alerts and mobile interface refinements; improved admin controls and report-generation tools

P4 — Advanced Air Quality Monitoring System using IoT and Sensor Technology (2025, IDCIoT)

Authors: Rohan Garg, Ankur Kumar, Dr. R. Dayana, Sakcham Singh (SRM Institute of Science and Technology, Chennai, India)

Field	Detail
Research problem	Lack of real-time/live pollution tracking in urban areas, either due to insufficient sensors or lack of detailed comparison of sensor accuracy and data transmission; existing systems limited to few parameters, reducing "prediction quality"
Application/domain	Indoor and outdoor monitoring, specifically framed around household safety near industrial/polluted areas ("within 15 m in all directions")
IoT architecture	DHT11, MQ135, BMP180 → ESP32 → Blynk cloud → LCD (local display) + web dashboard + mobile app + email notification
Sensors/hardware	DHT11 (temp/humidity), MQ135 (CO_2 as primary parameter; also ammonia, alcohol, benzene, smoke), BMP180 (barometric pressure)
Microcontroller/edge	ESP32 (explicitly chosen over Arduino for cost, Wi-Fi, low power)
Communication protocol	Wi-Fi (ESP32 to Blynk cloud); I2C used locally between ESP32 and 16×2 LCD
Data collection method	Continuous real-time sampling; displayed simultaneously on local LCD and remote dashboard/app; email alert triggered above thresholds
Dataset	Self-collected, real-world; three discrete short experiments (indoor household, household near industrial area, outdoor near industrial area), each a single-timestamp snapshot table, not a continuous logged dataset; no dataset name
Cloud/fog/edge	Cloud only: Blynk (dashboard, alerting)
AI/ML	**None implemented.** Explicitly proposed only as future scope: "Machine Learning can be deployed and trained with the different set of data... for the future prediction of air quality and early warning system."
What it monitors	CO_2 (ppm, via MQ135), temperature, pressure, humidity; rule-based classification (block diagram: <1200 ppm = Good, >1200 ppm = Bad, >2500 ppm = Danger/Alert)
Evaluation metrics	No formal accuracy/error metric; results reported as raw sensor value snapshots per site/time; a qualitative literature comparison table (Table IV) lists gaps in *other* cited works, not their own quantified evaluation
Main results	Household indoor: "Fresh Air" (859 ppm CO_2); household near industrial area: "Fresh Air" (1247 ppm CO_2); outdoor near industrial area: "Moderate Air" (1238 ppm CO_2, 32°C). System correctly differentiated locations qualitatively but reported no statistical validation.
Main contribution	Multi-parameter (gas + temperature + humidity + pressure) integration on a single low-cost ESP32 platform with simultaneous local (LCD) and remote (Blynk web/app/email) output
Limitations (explicit)	Tested only in a controlled environment; authors state "further experiments are necessary in environments that better mimic real atmospheric conditions"; MQ135 requires 60 s preheating before accurate readings; running two heated MQ sensors simultaneously can cause "unbalanced signal" due to power draw (noted for related work, and implicitly a shared MQ-sensor issue); no PM sensor; no calibration-to-PPM validation shown (only raw sensor "gas value" reported, not confirmed against a reference instrument)
Future work (explicit)	Add SI1145 (sunlight), PID (photoionization) sensor, DHT22, IR gas sensors; migrate to STM32 microcontroller to increase range/capability; deploy ML trained on collected data for future air-quality prediction and early-warning system

3. Comparative Table

Paper	Year	Problem addressed	Sensors	IoT hardware	Comm. protocol	IoT architecture	Cloud/Edge/Fog	Dataset	AI/ML	Pre tas
P1 Vayu Mitra	2025	Expensive/stationary govt AQ stations; need low-cost portable community system	MQ135, MQ131, MQ7, PMS7003, DHT11	NodeMCU ESP8266 / ESP32 (paper inconsistent)	Wi-Fi	Sensors→MCU→ThingSpeak+Blynk→User alerts	Cloud only	Self-collected, real-time, no dataset name	None	Thr haz
P2 Hybrid	2023	Combine static+mobile IoT sensors for better AQ monitoring **and** prediction	DHT11/DHT22, MQ-6, Honeywell HPMA115S0	Raspberry Pi 3B+ (+MCP3008 ADC)	Wi-Fi	Sensors→RPi→ThingSpeak→offline RFR (client PC)→mobile app	Cloud (ThingSpeak) + offline PC processing	Self-collected via ThingSpeak export; not public	Random Forest Regression	Reg pre hu PM me
P3 ACHTECO	2025	No official AQMS in Cabuyao City; Clean Air Act compliance gap	MQ2, MQ7, MQ135, DHT22, NEO-6MV2 GPS	ESP32 (ESP8266 mentioned once, inconsistent)	Wi-Fi	Sensors+GPS→ESP32→centralized DB→web app (map/history/admin)	Cloud/centralized web DB	Self-collected, 2 field sites, 10-min intervals	None (AQI via EPA formula)	AQ cor rep
P4 Advanced AQMS	2025	Lack of real-time, multi-parameter urban pollution tracking	DHT11, MQ135, BMP180	ESP32	Wi-Fi (+I2C locally)	Sensors→ESP32→Blynk→LCD+web+app+email	Cloud only (Blynk)	Self-collected, 3 snapshot experiments, no dataset name	None (ML proposed as future work only)	Rul cla (Go

4. IoT Architecture Comparison

Pipeline per paper (Sensor → Processing/MCU → Communication → Gateway → Cloud/Edge → Application/User):

- **P1:** MQ135/MQ131/MQ7/PMS7003/DHT11 → ESP8266/ESP32 (contradictory) → Wi-Fi → *(no separate gateway; MCU connects directly)* → ThingSpeak + Blynk → Dashboard + SMS/push alerts to user
- **P2:** DHT11/DHT22, MQ-6, HPMA115S0 → MCP3008 (ADC) → Raspberry Pi 3B+ → Wi-Fi → *(no gateway)* → ThingSpeak (cloud) → offline RFR model on client PC → MIT App Inventor mobile app
- **P3:** MQ2/MQ7/MQ135, DHT22, NEO-6MV2 GPS → ESP32 → Wi-Fi → *(no gateway)* → centralized web-hosted DB → web app (map, history, admin) for public + CENRO
- **P4:** DHT11, MQ135, BMP180 → ESP32 → Wi-Fi → *(no gateway)* → Blynk cloud → LCD (local) + web dashboard + mobile app + email

Which architecture is simplest? P4 and P1 are the simplest — a single MCU (ESP32) directly polling 3 analog/digital sensors and pushing to Blynk, with no separate ADC board, no GPS, no ML pipeline, and no custom backend.

Which is most suitable for simulation? P1's and P4's architectures (ESP32/ESP8266 + MQ sensors + DHT + Wi-Fi + ThingSpeak/Blynk) are most reproducible in **Wokwi**, which natively supports ESP32, DHT22/DHT11, and generic analog gas-sensor emulation, plus a virtual Wi-Fi connection to real cloud endpoints like ThingSpeak. P2's Raspberry Pi + MCP3008 + offline ML pipeline is harder to simulate purely in Wokwi (Raspberry Pi support is limited) and would need a hybrid simulation + Python ML script approach. P3's GPS + Haversine-based location validation adds simulation complexity (NEO-6MV2 GPS is supported in Wokwi but the location-testing methodology is elaborate).

Which communication protocols are easiest to reproduce? Wi-Fi + HTTP/MQTT-based cloud push (as in ThingSpeak/Blynk, used by all four papers) is by far the easiest to reproduce, since Wokwi ESP32 boards support real internet connectivity to these platforms during simulation. None of the papers use LoRa/ZigBee/BLE as the primary link, which is fortunate for reproducibility, but also means the literature offers **no protocol diversity** to build on if the project wants to explore, e.g., LPWAN range/power trade-offs.

Which architecture is most realistic for a student IoT project? P1/P4-style **ESP32 + MQ/DHT sensors + Wi-Fi + cloud dashboard** is the most realistic: single low-cost board, well-documented libraries, native Wokwi support, and a working cloud target (ThingSpeak, free tier). P3's approach (GPS + custom backend + full web app + rigorous statistical validation) is realistic in scope for a 5-person team but requires more software-engineering effort (backend + frontend + DB) than IoT-networking effort.

Which components would be unnecessary complexity for the project?

- P2's Raspberry Pi + separate ADC chip (MCP3008) — unnecessary if using ESP32, which has native ADC pins.
- P3's GPS module and Haversine distance validation — valuable for a mobile/multi-site deployment claim, but unnecessary complexity if the project's contribution is prediction/ML rather than location verification.
- Custom full-stack web backends (P3) — a lot of non-IoT software engineering effort relative to the "IoT Networks, Architectures and Applications" course focus; ThingSpeak/Blynk (as used by P1, P2, P4) offloads this.

---

5. AI/ML Analysis

1. **Which papers use AI/ML?** Only **P2**. P1, P3, and P4 use zero AI/ML — P1 and P4 use static/dynamic threshold rules, and P3 uses a deterministic EPA AQI formula.
2. **Which algorithms are used? Random Forest Regression (RFR)** (P2 only).
3. **Input features?** Sensor readings (temperature, humidity, PM2.5, PM10, CO, LPG, methane, alcohol) — one field designated as the dependent variable per model run, others as independent variables (features).
4. **Prediction/classification target?** Regression prediction of individual pollutant/environmental values (not a classification of "safe/unsafe," and not a composite AQI prediction).
5. **Datasets used?** Only P2's own self-collected ThingSpeak-exported dataset; no public dataset is used by any of the four papers.
6. **Performance reported?** RMSE, ranging 16.535–29.507 for LPG, methane, CO, and alcohol — described by the authors themselves as "considerable" and indicative of declining accuracy from data abnormalities. No RMSE values are given for temperature/humidity/PM2.5/PM10 predictions in the provided text (Fig. 3 shows plots but the discussed numeric RMSE values focus on the four gases).
7. **Is ML actually integrated with the live IoT system, or performed offline? Offline.** P2 explicitly states the dataset was "obtained from ThingSpeak" and "used on a client-side PC to employ random forest regression (RFR) for prediction." The paper does state the *intent* to incorporate "the trained models into the IoT device's software system" and develop "real-time processing algorithms," but the Results section only reports the offline training/evaluation — there is no evidence in the paper of the trained model running on-device or making live predictions from streaming sensor data during actual operation.
8. **Limitations in their AI/ML approach?** Large/unstable RMSE for several pollutants; a single offline RFR algorithm tried, no comparison against alternative models (e.g., LSTM, XGBoost, SVR) despite the related-work section citing LSTM/GRU/deep-learning approaches used by others; no cross-validation described; no discussion of feature importance, overfitting checks, or outlier handling despite noting "abnormality" in the data.
9. **Opportunity to integrate real-time IoT data with ML prediction? Yes — clearly, and this is the single strongest, most literature-supported gap.** Across all four papers, sensing infrastructure is mature and reliable (P3 in particular shows sensors can achieve 99%+ accuracy vs. professional devices), but not one paper demonstrates a closed loop where live streaming sensor data is fed into a model for real-time inference and alerting. P1 and P4 explicitly name "predictive analytics"/"ML deployment" as *future* work rather than achieved work; P2 gets closest but stops at offline batch training.

Conclusion for Part 4: AI/ML is present in only one of four papers, and even there it is not integrated into the live IoT pipeline. This is not a "no evidence yet" statement to be cautious about — it is a **repeated, explicitly acknowledged gap**: P1 states its own baseline system has "no predictive analysis or AI-based data processing" as a named limitation; P4 explicitly defers ML to future work; P2's ML is confirmed offline by its own methodology description. This constitutes real evidence of a research gap, not a manufactured one.

---

6. Research Gaps

For each gap: **Gap / Evidence / Why it matters / Possible solution / Difficulty / Novelty**

Gap 1 — Lack of integration between real-time IoT sensing and intelligent (ML) prediction

- **Evidence:** P1 explicitly lists "no predictive analysis or AI-based data processing" as a limitation and lists predictive analytics as future work. P4 explicitly defers ML to future work. P2 implements ML but only offline/batch on exported data, not on live streaming data. P3 uses a deterministic formula (AQI), not ML, despite being the most rigorously validated system.
- **Why it matters:** Threshold/rule-based alerting (used by 3 of 4 papers) reacts only after a pollutant crosses a fixed limit; it cannot forecast an approaching hazardous trend or personalize thresholds to local baseline conditions.
- **Possible solution:** Stream simulated/live sensor data into a lightweight model (e.g., a regression or shallow LSTM) running either on-device (edge) or in a lightweight cloud function, producing short-horizon AQI forecasts alongside the current-value display.
- **Difficulty for student project:** Medium — Wokwi can emulate the sensors/MCU; the ML model can be trained offline on a public dataset and deployed via a simple inference script or TinyML runtime; the "real-time" claim can be satisfied by feeding a live simulated stream into the trained model.
- **Potential novelty: Medium–High** (directly closes a gap named or implied in 3 of 4 papers).

Gap 2 — Absence of public/standardized datasets and multi-site/long-duration data collection

- **Evidence:** None of the four papers use a public/named dataset. P1: no dataset details at all. P2: self-collected, exported from ThingSpeak, no public source. P3: 2 field sites, 10-minute intervals — real but narrow. P4: 3 single-timestamp snapshots, explicitly called out by the authors as needing testing "in environments that better mimic real atmospheric conditions."
- **Why it matters:** Small, short, single-team datasets limit model generalizability and make it hard for future work (including this course's own literature-review continuation) to benchmark or reproduce results.
- **Possible solution:** Combine a public dataset (e.g., a Kaggle/UCI-style air-quality dataset) for offline model pretraining with a simulated multi-node Wokwi deployment for the live/IoT-specific demonstration, explicitly documenting this hybrid approach as compensating for the field's lack of shared data.
- **Difficulty:** Easy–Medium.

- **Potential novelty:** Low–Medium on its own, but Medium–High when combined with Gap 1 (i.e., "public-dataset-pretrained model fused with simulated live IoT stream").

Gap 3 — Cloud-only processing; no edge/on-device intelligence

- **Evidence:** All four papers process data in the cloud (ThingSpeak/Blynk/custom DB) or, in P2's case, on a client-side PC — never on the constrained IoT device itself. P1 names "edge computing... to reduce cloud dependency" as future work, confirming the authors themselves see this as unaddressed.
- **Why it matters:** Cloud-only processing adds latency and creates a hard dependency on constant connectivity (P1 explicitly lists Wi-Fi dependency as a limitation); this conflicts with real-time hazard alerting where local, low-latency response matters.
- **Possible solution:** Implement simple on-device threshold logic *plus* a lightweight edge-deployed model (e.g., TensorFlow Lite Micro on ESP32, or a rule-augmented statistical anomaly detector) so that alerting doesn't depend on round-trip cloud latency.
- **Difficulty:** Medium–Hard (TinyML on ESP32 in Wokwi is feasible but nontrivial for a first IoT project).
- **Potential novelty:** Medium.

Gap 4 — No anomaly detection; only static or manually-set thresholds

- **Evidence:** P1's alerting is threshold-based against WHO-derived limits. P3 computes AQI via the fixed EPA formula and flags exceedance of accepted ambient values. P4 uses fixed CO_2 bands (<1200 / >1200 / >2500 ppm) hard-coded in the block diagram. No paper implements statistical or ML-based anomaly/outlier detection — notably, P2 itself observed unexplained "abnormality" in its own data causing RMSE spikes, but did not attempt to detect or flag it algorithmically.
- **Why it matters:** Static thresholds cannot catch sensor drift, sudden spikes, or contextual anomalies (e.g., a reading that is "normal" in absolute terms but abnormal for that time/location); P2's own unresolved RMSE anomaly is direct evidence this is an open problem in this exact line of work.
- **Possible solution:** Add a simple anomaly-detection layer (e.g., z-score/IQR-based or Isolation Forest) on top of the existing threshold logic, flagging statistically unusual readings independent of fixed pollutant limits.
- **Difficulty:** Easy–Medium.
- **Potential novelty:** Medium.

Gap 5 — Limited sensor scope; PM largely absent or add-on only

- **Evidence:** P3 and P4 have **no PM sensor at all**, despite both papers naming particulate matter as a key pollutant in their own introductions/literature reviews. P1 explicitly frames PM sensing as an *upgrade* over its own prior baseline (PMS7003 added to address the baseline's stated PM gap). P2 includes PM via Honeywell HPMA115S0.
- **Why it matters:** PM2.5/PM10 are repeatedly cited across all four papers' introductions as major health-relevant pollutants, yet half the papers omit them — a real, repeated, paper-acknowledged inconsistency between stated motivation and implemented sensor suite.
- **Possible solution:** Include a PM sensor (e.g., PMS5003/7003 or SPS30, both Wokwi-compatible or reasonably mockable) as a baseline requirement, not an afterthought.
- **Difficulty:** Easy.
- **Potential novelty:** Low on its own (this is a hardware completeness fix, not a research contribution) — but worth doing as part of a stronger baseline for Gap 1/3/4.

Gap 6 — No scalability/multi-node demonstration

- **Evidence:** P1: single unit implied, no node count stated. P2: "hybrid" of stationary + mobile sensors, but described at small scale with no stated number of nodes. P3: exactly 2 physical units at 2 fixed locations. P4: single unit tested in 3 different physical locations sequentially (not simultaneously). None demonstrates a multi-node network with data fusion across nodes, and P1 itself states the system "currently operates at a small scale."
- **Why it matters:** Real air-quality monitoring value comes from spatial coverage (multiple simultaneous nodes), which none of the four papers demonstrate.
- **Possible solution:** Simulate 3–5 virtual nodes in Wokwi/Proteus feeding a shared dashboard, demonstrating spatial comparison/heatmapping — feasible without needing physical hardware for each node.
- **Difficulty:** Medium.
- **Potential novelty:** Medium.

Explicitly NOT treated as gaps (to avoid manufacturing gaps):

- *Security/privacy* is not discussed in any of the four papers, but since it is not raised, contradicted, or flagged as a limitation by any paper, and the papers give no evidence either way, this is a **generic absent feature, not a literature-supported gap —** it is noted in Part 7 (next 11 papers) as something to check for, not asserted as a gap here.
- *Energy efficiency* is not analyzed quantitatively in any paper (no battery-life or power-consumption figures reported), so while it's plausible as a real-world concern, the four papers give no direct evidence (positive or negative) to build a gap claim on. Also deferred to the "what we still need to find" list.

---

7. Common vs. Unique vs. Missing Features

A. Commonly done across all/most papers

- Wi-Fi-connected microcontroller (ESP32/ESP8266 family) as the central processing unit
- MQ-series analog gas sensors as the primary gas-detection method
- DHT-series sensor for temperature/humidity
- Cloud dashboard for real-time visualization (ThingSpeak and/or Blynk in 3 of 4 papers; custom web app in the 4th)
- Threshold-based alerting (SMS/push/email) rather than predictive alerting
- Self-collected, non-public, relatively short-duration datasets
- No LoRa/ZigBee/MQTT — all rely on Wi-Fi + HTTP-style cloud APIs

- Explicit framing around health/regulatory motivation (WHO limits, Clean Air Act, "critical environmental challenge")

B. Done by only one or two papers

- Machine learning / predictive modeling (P2 only, and only offline)
- GPS-based location verification and Haversine-based accuracy testing (P3 only)
- Formal statistical validation (accuracy %, paired t-test, stress testing, UAT scoring) (P3 only)
- Hybrid stationary + mobile sensor deployment (P2 only)
- Barometric pressure sensing (P4 only, via BMP180)
- Particulate matter sensing (P1 and P2 only; absent in P3, P4)
- Local on-device display via LCD (P4 only)
- Custom (non-commercial) backend/database and full web application with role-based admin access (P3 only)

C. Apparently missing / insufficiently explored across all four papers

- Real-time (streaming) ML inference integrated into the live IoT pipeline
- Anomaly/outlier detection independent of fixed thresholds
- Edge/on-device processing (TinyML or even simple edge filtering beyond basic smoothing)
- Any public or shared dataset use, or dataset publication for reproducibility
- Multi-node, simultaneous, networked deployment with spatial data fusion
- LPWAN protocols (LoRa/LoRaWAN, ZigBee) — none tested, so no comparative evidence on range/power trade-offs
- Quantified energy/power-consumption analysis
- Security/privacy treatment of transmitted environmental/location data
- Long-duration (multi-week/seasonal) monitoring and trend analysis
- Localized/region-specific model calibration (e.g., adjusting thresholds or models for local baseline pollution levels rather than generic WHO/EPA limits)

---

8. Proposed Project Directions

Direction 1 — Real-Time Edge-Assisted AQ Forecasting (Primary Recommendation)

- **Project concept:** Simulated multi-sensor IoT node streams live readings; a lightweight regression/LSTM model — pretrained offline on a public AQ dataset and fine-tuned on simulated live data — runs prediction (short-horizon AQI forecast) either on-device or at a fog layer, closing the gap where P1/P4 defer ML to future work and P2 only does it offline.
- **IoT components:** ESP32 (Wokwi), simulated gas + PM + DHT sensors, Wi-Fi
- **Sensors:** MQ135, MQ7, DHT22, PMS7003/SPS30 (simulated)
- **Communication protocol:** Wi-Fi + MQTT or HTTP to a lightweight broker/dashboard
- **Simulation platform:** Wokwi (hardware/firmware) + Python (model training, offline, using a public dataset)
- **Cloud/edge component:** Edge inference on ESP32 (TensorFlow Lite Micro) for lightweight rule+anomaly logic; heavier forecasting model can run at a "fog" node (e.g., a laptop/Raspberry Pi acting as gateway) to keep the MCU workload realistic
- **AI/ML component:** Regression or simple LSTM for short-horizon pollutant/AQI forecasting; anomaly detector (z-score/IQR) for outlier flagging
- **Expected output:** Live dashboard showing current + forecasted AQI, with anomaly flags distinct from simple threshold breaches
- **Research gap addressed:** Gap 1 (real-time ML integration), Gap 3 (edge processing), Gap 4 (anomaly detection)
- **Implementation difficulty:** Medium
- **Expected research novelty:** High

Direction 2 — Multi-Node Simulated Sensor Network with Spatial AQ Mapping

- **Project concept:** 3–5 simulated nodes (Wokwi instances) at different virtual "locations" feed a shared dashboard that fuses readings into a spatial heatmap, directly addressing the single/dual-node limitation shared by all four papers.
- **IoT components:** Multiple ESP32 nodes, shared Wi-Fi network (simulated)
- **Sensors:** MQ135, MQ7, DHT22 per node
- **Communication protocol:** MQTT (publish/subscribe across nodes — a protocol none of the 4 papers used, adding genuine protocol-level novelty)
- **Simulation platform:** Wokwi (multi-instance) + a lightweight MQTT broker (e.g., Mosquitto) + a web dashboard (Node-RED or simple custom app)
- **Cloud/edge component:** Cloud aggregation only (fog/edge optional extension)
- **AI/ML component:** Optional — simple spatial interpolation (e.g., inverse-distance weighting) rather than full ML, to keep scope realistic
- **Expected output:** A live spatial AQI map across simulated locations, with per-node trend charts
- **Research gap addressed:** Gap 6 (scalability), and partially Gap 2 (broader synthetic "dataset" from multiple simulated nodes)
- **Implementation difficulty:** Medium
- **Expected research novelty:** Medium

Direction 3 — Anomaly-Aware Alerting Layer on a Vayu-Mitra/ACHTECO-style Baseline

- **Project concept:** Replicate the well-validated P3-style pipeline (multi-gas + PM + calibration) but add a statistical anomaly-detection layer on top of EPA/WHO-threshold alerting, directly testing whether anomaly detection catches events that fixed thresholds miss (referencing P2's own unresolved data-abnormality finding).
- **IoT components:** ESP32, MQ135/MQ7/MQ131, PMS7003, DHT22

- **Sensors:** As above
- **Communication protocol:** Wi-Fi + HTTP to ThingSpeak (reproducing P1's easiest-to-simulate stack)
- **Simulation platform:** Wokwi + ThingSpeak (free tier)
- **Cloud/edge component:** Cloud (ThingSpeak) for storage; simple Python/Node script for anomaly-detection layer
- **AI/ML component:** Lightweight anomaly detection (Isolation Forest or z-score), not full prediction — keeps scope tight and achievable
- **Expected output:** Dashboard distinguishing "threshold breach" alerts from "statistical anomaly" alerts, with a comparison of how often each fires and why
- **Research gap addressed:** Gap 4 (anomaly detection), partially Gap 5 (complete sensor suite)
- **Implementation difficulty:** Easy–Medium
- **Expected research novelty:** Medium

Direction 4 — Public-Dataset-Pretrained + Simulated-Live-Data Fusion Model

- **Project concept:** Address the "no public dataset" gap directly: pretrain a prediction model on a public AQ dataset (e.g., UCI/Kaggle), then fuse it with a simulated live Wokwi sensor stream to demonstrate transfer from public data to a local/simulated deployment — explicitly filling Gap 2 while still delivering a working prediction pipeline (like Direction 1, but with public-data pretraining foregrounded as the contribution).
- **IoT components:** ESP32, MQ135, DHT22
- **Sensors:** MQ135, DHT22 (minimal set, kept simple since the ML/data-fusion angle is the novelty, not the hardware breadth)
- **Communication protocol:** Wi-Fi + HTTP/MQTT
- **Simulation platform:** Wokwi + Python (offline pretraining + live inference script)
- **Cloud/edge component:** Cloud-hosted model inference (simple Flask/FastAPI endpoint) consuming the live simulated stream
- **AI/ML component:** Regression model pretrained on public dataset, fine-tuned/evaluated on simulated local stream; explicit before/after comparison of public-data-only vs. fused-model accuracy
- **Expected output:** Comparative accuracy report (public-only model vs. fused model) plus live dashboard
- **Research gap addressed:** Gap 2 (dataset absence) and Gap 1 (real-time ML integration)
- **Implementation difficulty:** Medium
- **Expected research novelty:** Medium–High

Team-of-5, one-semester feasibility note [Inference]: Directions 1 and 3 are the most balanced for a 5-person team: they reuse well-documented hardware/cloud stacks from the literature (minimizing hardware risk) while adding a genuinely under-addressed ML/anomaly-detection layer (maximizing novelty-to-effort ratio). Direction 2 is attractive for demonstrating "IoT Networks and Architectures" course learning outcomes specifically (multi-node, pub/sub protocol) but has less research-paper-style novelty. Direction 4 is the most "research-flavored" but carries dataset-compatibility risk (a public dataset's sensor set/units may not match simulated hardware cleanly) and would need early validation before committing.

---

9. Recommended Project Architecture

Combining the strongest, most reproducible elements identified above:

This architecture is deliberately closest to **P1's** stack (simplest, most Wokwi-reproducible) plus **P3's** completeness (PM sensor, full sensor suite) plus a genuinely new layer that none of the four papers deliver: a prediction/anomaly service that consumes the live stream rather than operating offline or not at all.

10. Final Project Title Suggestions

1. **"Edge-Assisted Real-Time Air Quality Forecasting and Anomaly Detection using a Simulated Low-Cost IoT Sensor Network"**
2. **"A Hybrid Public-Dataset-Pretrained and Live-Stream-Fused Machine Learning Model for Real-Time IoT Air Quality Prediction"**
3. **"Design and Simulation of a Multi-Node IoT Air Quality Monitoring Network with Statistical Anomaly Detection for Enhanced Early Warning"**

---

11. Key Questions We Should Investigate in the Next 11 IEEE Papers

1. Are there IEEE papers (2021–2026) that successfully deploy ML **on-device or at the edge** (not offline) for air-quality prediction, and what latency/accuracy trade-offs do they report?
2. What public air-quality datasets (UCI, Kaggle, government open-data portals, or IEEE DataPort) exist that are compatible with MQ-series/PM-series sensor outputs, so a "public data + simulated live data" fusion approach is technically sound?
3. Are there published comparisons of MQTT vs. HTTP vs. LoRaWAN specifically for air-quality IoT nodes, with power/latency/range data our four papers didn't provide?
4. Has any paper implemented and evaluated anomaly detection (statistical or ML-based) specifically for air-quality sensor streams, and how does it compare to fixed-threshold alerting in practice?
5. What multi-node / networked AQ monitoring deployments exist in the IEEE literature, and how do they perform data fusion or spatial interpolation across nodes?
6. Do any papers report quantitative energy-consumption or battery-life figures for ESP32-class AQ nodes, which none of our four papers provided?
7. Is there literature on calibrating MQ-series sensors against reference-grade instruments with reported calibration curves/error bounds (beyond P3's accuracy percentages), that we could adopt for our own calibration methodology?
8. Are there papers addressing security/privacy of transmitted air-quality/location data (e.g., encrypted MQTT, authentication) that would let us responsibly add a security dimension if time allows?
9. What Wokwi/Proteus-specific implementations of PM sensors (PMS5003/7003/SPS30) exist in published or open-source IEEE-adjacent work, to validate our simulation approach is realistic?
10. Are there papers that explicitly compare rule-based/threshold alerting against ML-based prediction on the *same* dataset, giving us a benchmark for how much value the ML layer actually adds?
11. Do any recent (2024–2026) papers address the same regulatory-compliance angle as P3 (ACHTECO) but combine it with a predictive/ML layer — i.e., has anyone already done exactly our proposed direction, which we need to know to correctly position our novelty claim?

WHAT WE STILL NEED TO FIND

- A **public air-quality dataset** compatible with common low-cost sensor outputs (for offline pretraining).
- At least one paper demonstrating **genuine on-device/edge ML inference** for AQ data (to benchmark feasibility and avoid re-deriving edge-deployment methodology from scratch).
- Quantitative **power/energy consumption** figures for ESP32-class AQ nodes (none of the 4 papers report this).
- A clear picture of whether **anomaly detection for AQ IoT data** has already been done in recent literature (to confirm Gap 4 is still open and correctly scope our novelty claim).
- Evidence on **LoRa/MQTT/ZigBee comparative performance** for AQ monitoring (all 4 papers used Wi-Fi only, so we have no protocol-tradeoff evidence yet).
- Confirmation of whether **any 2024–2026 paper has already combined a Clean-Air-Act/compliance-style system (like P3) with real-time ML —** if so, we need to differentiate; if not, this strengthens our novelty claim.
- Any published **security/privacy treatment** of AQ-IoT data transmission, since none of our 4 papers touch this at all.
- Verification of whether **multi-node spatial fusion/mapping** has been done at a similarly small (student-project) scale elsewhere, to calibrate how much novelty Direction 2 actually carries.